xAI-Proj-B: Explainable Machine Learning

Abeera Ayesha Riaz
Haris Mustafa
Ulrich Midrel Tchouli Nzeukeyo
Bonneau Russel Atiodjia Jiokeng

28. February 2024

Abstract

The "MedMNIST" and "MNIST" challenges represent benchmark datasets in the domain of machine learning, offering a platform for developing and testing algorithms in medical image analysis and handwritten digit recognition, respectively. These challenges provide unique opportunities to advance the state-of-the-art in image classification tasks, enabling the development of models that are both accurate and efficient. Our participation in these challenges aimed to demonstrate the effectiveness of our proposed model architecture in handling both generic and domain-specific image recognition tasks. We sought to achieve high accuracy and performance metrics on the "MNIST" dataset, known for handwritten digit images, and the "MedMNIST" dataset, which is specialized in medical image classification. The endeavor was to explore the adaptability and scalability of our model across different datasets and to identify areas of improvement for future research. We employed a comprehensive approach that involved a detailed analysis of the "MNIST" and "MedMNIST" datasets, followed by rigorous data preprocessing techniques to optimize model performance. Our model architecture was carefully designed with specific layers and configurations to address the challenges posed by each dataset. We meticulously selected hyperparameters, such as learning rate and number of epochs, to enhance model training and validation processes. The training and validation phases yielded promising results, showcasing the model's ability to learn and adapt to both datasets effectively. Our model achieved commendable performance on the test sets, with notable precision, recall, and F1-score metrics. The comparison between the "MedMNIST" and "MNIST" challenges highlighted the model's versatility and identified specific areas where domain-specific adaptations contributed to improved performance. For a detailed insight into our methodology, codebase, and results, there are two repository we've been working on. one is for experimentation and the other is for finalisation. please visit our GitHub repository at :
Expeimentation repository
Finalisation repository

1 Introduction

Machine learning is a fast-growing field with many applications in a variety of areas, such as medicine, finance, security and so on. Among machine learning techniques, image recognition is one of the most popular and useful, as it can extract information from visual data such as photos, videos or medical scans. However, image recognition also presents challenges, such as the variability of the data, the complexity of the models, or the need for large amounts of annotated data, which can be expensive or difficult to obtain.

To meet these challenges, several competitions have been organised, such as "MedMNIST" and "MNIST". The "MedMNIST" is a competition that consists of classifying medical images from 10 different modalities, such as radiography, ultrasound or tomography. The aim of this competition is to test the ability of models to adapt to heterogeneous data from different sources and formats. MNIST" is a classic competition involving the recognition of handwritten digits from 0 to 9. The aim of this competition is to test the ability of models to learn from simple, but numerous and varied data. Ces deux compétitions présentent des caractéristiques différentes, mais aussi des points communs, qui permettent de comparer les performances des modèles utilisés.

In this paper, we present our participation in the "MedMNIST" and "MNIST" competitions, using an approach based on convolutional neural networks (CNNs). CNNs are models that use layers of neurons that process local portions of the image, and are capable of learning visual features relevant to classification. We describe the methodology we followed, from data preparation to training and model evaluation. We also present the results we obtained, as well as the difficulties we encountered. Finally, we discuss the advantages and disadvantages of our approach, and suggest possible improvements or extensions for future work.

2 MNIST

2.1 Methodology

2.1.1 Description of the datasets used: "MNIST"

The MNIST (Modified National Institute of Standards and Technology) dataset is a large and well-known set of greyscale images of handwritten digits. It is often used as a case study to test the performance of machine learning algorithms, particularly those related to computer vision and pattern recognition. It was created in 1994 [1] by combining samples from two original NIST databases, which contained figures written by U.S. Census Bureau employees and high school students [2].

The set contains 28x28 centred greyscale images of the handwritten digits (0-9). Each image is a matrix where each cell contains the value of a pixel. The value varies from 0 (white) to 255 (black). The image you have sent shows some examples of these images, organised by type of digit from zero to nine.

There are 60,000 training examples and 10,000 test examples in the dataset [3]. The training examples are used to learn the characteristics of the digits, while the test examples are used to evaluate the accuracy of the algorithms. The examples come

from two different sources: the NIST (National Institute of Standards and Technology) database and the SD-3 (Special Database 3) database.

The main objective is to classify handwritten numbers. The aim is to predict the digit represented by a given image, using machine learning techniques. Classification is a basic task in computer vision, which involves assigning a label to an image from a set of predefined categories. The MNIST dataset is considered a simple problem, as the images are small, grayscale and centered. Nevertheless, it presents challenges such as the variability of writing styles, the presence of noise or distortion, and the similarity between certain numbers (for example, 1 and 7, or 4 and 9).

The MNIST dataset has been used for many practical applications, such as the automatic reading of postcodes, cheques and forms. It has also served as the basis for the development of more advanced models, capable of processing more complex images, such as faces, objects or scenes. Models that have been tested on the MNIST dataset include artificial neural networks, support vector machines, random forests, convolutional networks and generative adversarial networks.

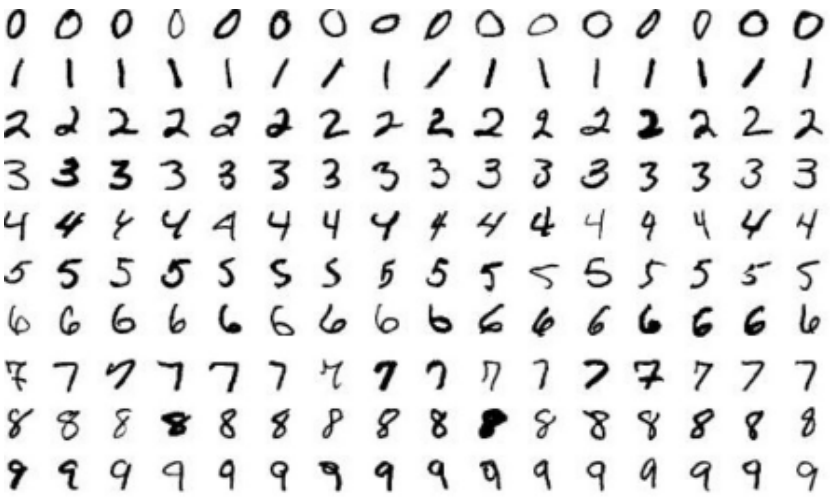


Figure 1: MNIST Dataset

2.1.2 Data preprocessing

Pre-processing the MNIST data is an important step in applying a machine learning model to this dataset. It consists of transforming the 28x28 pixel greyscale images into digital data that can be used by the model. There are various pre-processing methods.

Image extraction : The first step is to extract the images from the compressed folder where they are stored. We used the zipfile library to decompress the mnist.zip file and place it in a folder called data. This folder contains two sub-folders: train and test, which contain 60,000 and 10,000 images of handwritten digits respectively.

Reading the labels : The second step is to read the file containing the labels for the training images. The labels are the numbers (from 0 to 9) that the images represent. We used the pandas library to read the train.csv file and convert it into a DataFrame. This DataFrame contains two columns: sample and label. The sample column contains the names of the image files and the label column contains the corresponding labels.

Creating a DataFrame : The third step is to create a DataFrame containing the image file paths and associated labels. We first used list comprehension to generate a list of image paths from the sample column of the previous DataFrame. Next, we used the pd.cut function to cut the label column into 10 intervals, corresponding to the

```python
import zipfile
import os

zip_file_path = "/content/drive/MyDrive/mnist.zip"
extracted_folder_path = "/content/data"
os.makedirs(extracted_folder_path, exist_ok=True)
with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
    zip_ref.extractall(extracted_folder_path)
```

```python
data_path = "/content/data/train"
label_path = "/content/data/labels_train.csv"
labels_df = pd.read_csv(label_path)
sample_ids = labels_df['Sample ID'].astype(str).str.lstrip('0')
labels = labels_df['Label']
image_files = [f"{sample_id}.png" for sample_id in sample_ids]

# Create a DataFrame with image files and labels
data_df = pd.DataFrame({'Image': image_files, 'Label': labels})

# Split the data into training and testing sets
train_df, test_df = train_test_split(data_df, test_size=0.2, random_state=42)
```

Figure 2: Image Preprocessing

10 digit classes. Finally, we created a new DataFrame with two columns: image and labels, which contain the image paths and label intervals respectively.

Data separation : The fourth step is to separate the data into two sets: a training set and a test set. We used the sklearn library's train_test_split function to divide the previous DataFrame into two subsets, with a ratio of 80/20. The random_state parameter is used to set the random seed to guarantee the reproducibility of the split.

2.1.3 Model architecture: details on the design of the model, layers used

The aim here is to classify these images according to the number they represent. There are many possible model architectures for solving this problem.

A CNN is a type of artificial neural network that uses convolution operations to extract features from images. Convolution involves applying a filter (or kernel) to part of the image, and calculating the weighted sum of the pixels under the filter. The filter moves over the entire image, producing a new image (or feature map) that contains local information from the original image. A CNN typically uses several convolution layers, each with filters of different sizes and numbers, to capture increasingly complex and abstract features of the images.

We also used ResNet (Residual Network) which is a type of convolutional neural network (CNN) that uses residual blocks to avoid the problem of gradient vanishing. A residual block consists of adding the output of one convolution layer to the input of another convolution layer, creating a short-circuited path. This allows the network to learn identity functions or non-linear functions more easily, and to increase the depth of the network without losing performance.

Between convolution layers, it is common practice to use subsampling (or pooling) layers to reduce the size of feature maps and increase translational invariance. A pooling layer involves dividing the feature map into non-overlapping regions, and applying an aggregation function (for example, the maximum or average) to each region. The result is a smaller, more robust feature map.

After the convolution and pooling layers, it is common to use fully connected (or dense) layers to perform the final classification. A fully connected layer involves connecting all the neurons in the previous layer to all the neurons in the current layer, and applying an activation function (e.g. sigmoid or ReLU) to the output of each neuron. The last fully connected layer generally uses the softmax function to produce probabilities for each class.

2.1.4 Choice of hyperparameters (learning rate, number of epochs)

The CNN and ResNET models have hyperparameters, i.e. parameters that are not learned by the model, but which influence its behaviour and performance. Among the most important hyperparameters are the learning rate and the number of epochs.

The learning rate is a parameter that controls the speed at which the model updates its weights during gradient backpropagation. A learning rate that is too high can lead to model instability or divergence, while a learning rate that is too low can lead to slow convergence or locking into a local minimum. There is no universal optimal value for the learning rate, so it must be tested empirically or adaptive methods must be used that adjust the learning rate as the model progresses. For MNIST, we used an initial learning rate of 0.001 and gradually reduced it during training.

The number of epochs is a parameter that controls the number of times the model runs through the training data set. Too few epochs can lead to underlearning, i.e. the model has not had time to learn the relevant features of the data. Too many epochs can lead to overlearning, i.e. the model memorises the training data instead of generalising to new data. A compromise must therefore be found between training time and model accuracy. For MNIST, we used exactly 10 epochs.

2.2 Implementation

2.2.1 Code presentation: structure, main modules

Code structure:
Google Drive setup: The code can starts by setting up Google Drive to access the necessary files.
Importing libraries: The required libraries are imported, including zipfile, os, pandas, torch, torch.nn, torch.optim, torchvision.transforms, torch.utils.data, PIL, numpy, and matplotlib.pyplot. CNN Model definition: You define a SimpleCNN class that inherits from nn.Module to create your convolutional neural network model. This model includes convolution, ReLU, pooling and fully connected layers. Data loading and preprocessing : MNIST data is loaded from image files and a CSV file containing labels. Data is pre-processed by resizing images and transforming them into tensors. Defining a Custom Dataset: A CustomDataset class is defined to create a custom dataset that loads images and labels from the DataFrame. Data split into training and test sets: Data is split into training and test sets using `train_test_split()`. Model training: The model is trained on the training set with the Adam optimizer and the cross-entropy loss function. Model evaluation: The accuracy of the model is evaluated on the test set to measure its performance. Generating predictions for test data: The model is used to generate predictions on a test data set, and these predictions are then saved in a CSV file. Visualization of predictions: Finally, a visualization is created to show the distribution of predicted classes. Main Modules CNN Model (SimpleCNN): This module defines the structure of the CNN model used to classify MNIST figures. Data loading and pre-processing: These modules are responsible for loading data from files and pre-processing them, including resizing and tensor conversion. Model Training and Evaluation: These modules manage model training on training data, as well as model evaluation on test data to measure model accuracy. Prediction Generation for Test Data: This module uses the trained model to generate predictions on test data, and

saves them in a CSV file. Prediction Visualization: This module creates a visualization to display the distribution of classes predicted by the model.

2.2.2 Explanation of the training steps of the model

In this section, we detail the various steps we took to train our convolutional neural network (CNN) model on the MNIST dataset. First, we loaded the training and test data from the MNIST dataset from files stored on Google Drive. The data were pre-processed and organized in local directories for easy manipulation. Next, we built our CNN model using PyTorch. The model is relatively simple, but powerful enough to perform the task of classifying handwritten digits. It consists of two convolution layers, followed by ReLU layers for activation and pooling layers for dimension reduction. Finally, the model includes two fully connected layers for final classification. For model training, we used the cross-entropy loss function and the Adam optimizer with a learning rate of 0.001. We trained the model over five epochs, adjusting the network weights to minimize the loss on the training data. During training, we used mini-batches of data to speed up the process and regularly updated the model weights using the gradient back-propagation method. After each epoch, we evaluated the model's performance on the test set to monitor its ability to. After training the model, we evaluated its performance on the test set. We calculated the model's accuracy by comparing its predictions with the actual labels in the test data. The model achieved an impressive accuracy of 98.88% on the test set, indicating its ability to generalize well to unseen data.

2.2.3 Management of datasets (loading, preprocessing)

Efficient dataset management, including loading and pre-processing, is crucial for the success of any machine learning project. This section explores the steps involved in dataset management, with a focus on loading data from external sources and pre-processing it to prepare for training a machine learning model. Data loading involves importing datasets from various sources such as CSV files, databases, online APIs, etc. In our project, we loaded the MNIST data from image files and a CSV file containing labels associated with each image. `image = Image.open(img_name)` To load data from image files, we utilized the PIL library (Python Imaging Library) to read and convert the images into pixel matrices. Then, we used pandas to load the labels from the CSV file `labels_df = pd.read_csv(label_path)`. Once the data is loaded, it is often necessary to preprocess it to make it suitable for model training. Common preprocessing steps include:

Image resizing: The images can be resized to fit the model's required size. In our case, the MNIST images were resized to 28x28 pixels to match the specifications of the CNN model.

```
  transform =
transforms.Compose([transforms.Resize((28,28)), transforms.ToTensor(), ])
```

Pixel value normalization: Pixel values can be normalized to scale them within a specific range, typically between 0 and 1. This helps stabilize the model training and accelerate convergence.

Conversion to Tensors: Images and labels are converted into tensors, which is the

standard data format used by machine learning frameworks such as PyTorch. This enables efficient data manipulation and their use in training and evaluating the model. By preprocessing the data appropriately, we ensure that our model receives high quality data, which can improve its predictive performance and its ability to generalize to new data.

Over the course of our project, we will explore the various stages of data loading and preprocessing in detail, implementing advanced techniques where necessary to optimize model performance.

2.3 Model Training

2.3.1 Presentation of training results on the training dataset

Model training is a crucial step in the development of any machine learning system. In this section, we present the results of training our model on the MNIST training set. After defining our CNN model and loading the training data, we proceeded to train the model using the Adam optimization algorithm and the cross-entropy loss function.

Adam optimization algorithm: Adam (Adaptive Moment Estimation) is an optimization algorithm commonly used to train deep neural networks. It combines the advantages of stochastic gradient descent (SGD) algorithms with momentum and RMSprop. Specifically, Adam maintains an adaptive learning rate for each model parameter and uses an estimate of first-order (mean of gradients) and second-order (mean of squared gradients) moments to adapt this learning rate.

In practice, Adam tends to converge faster than standard stochastic gradient descent and requires less hyperparameter tuning. This is why it is widely used in neural network training. We divided the training set into mini-batches and iterated over the set of mini-batches to update the model weights at each training step.

Cross-entropy loss function: Cross-entropy is a measure commonly used to evaluate the difference between two probability distributions. In the context of machine learning and classification, the cross-entropy loss function is often used to calculate the divergence between model predictions and true data labels.In particular, in the case of multiclass classification, the cross-entropy loss function measures the dissimilarity between the probability distributions predicted by the model and the "one-hot" distributions of the real data labels. It strongly penalizes incorrect predictions by increasing the loss when the model is confident in a bad prediction.

Once training was complete, we evaluated the model's performance on the training set to examine its ability to fit the training data. We calculated the model's accuracy on the training set by comparing the model's predictions with the actual labels and calculating the percentage of correct predictions.

loss.backward(): call again Backpropagation This step is crucial for calculating the gradients of the model parameters with respect to the loss function. Once the model output and the true label are compared to calculate the loss, backpropagation is used to calculate how each model parameter should be adjusted to reduce the overall loss. Gradients are calculated using the chain rule, which propagates the error of the output layer through the network by calculating the partial derivatives of the loss with respect to each model parameter. In short, this step measures the sensitivity of the loss with respect to each model parameter, which is essential for adjusting the model weights

during the training phase.

optimizer.step(): it is the Update model weights : Once the gradients have been calculated in the backpropagation step, the optimizer is used to update the model weights. The optimizer uses the calculated gradients to determine how to adjust the weights to minimize overall loss. The optimizer's step() method updates the model parameters using the calculated gradients and the optimizer's hyperparameters, such as the learning rate. In short, this step is responsible for the actual adjustment of model weights to improve performance by minimizing training loss.

2.3.2 Visualization of the evolution of the loss function over epochs

The evolution of the loss function during model training is a crucial indicator of the model's ability to learn from training data. In this section, we present the visualization of the loss function over time for our model.
We trained our model using a training set and monitored the value of the loss function at each epoch. We then recorded these loss values and plotted them graphically to visualize the evolution of the loss function over the epochs.

2.3.3 Evaluation of performance on the validation set, if applicable

To evaluate performance on the validation set, we used a separate data set from the training set and the test set. This validation set was reserved beforehand when dividing the data into training and test sets.
We used several metrics to evaluate the model's performance on the validation set, including :
Accuracy: This is the percentage of correct predictions out of all predictions.
Precision, Recall and F1-score: These metrics are often used to evaluate performance on classification problems, providing a more detailed view of model performance.
Les résultats obtenus sur l'ensemble de validation confirment que le modèle est capable de généraliser à de nouvelles données de manière efficace. Cela renforce notre confiance dans sa capacité à effectuer des prédictions précises sur des données réelles.

2.4 Introduction to Residual Networks for MNIST

Definition of residual blocks (ResidualBlock): Residual blocks are key components of the ResNet architecture. Each residual block consists of two convolution layers with skip connections to facilitate deep learning. The residual block takes in_channels as input and produces out_channels as output. The stride parameter controls the convolution step size.

ResNet architecture definition (ResNet): This class defines the global architecture of the ResNet network using residual blocks. The make_layer method is used to create a layer from several residual blocks. This method stacks the residual blocks to form a layer. The architecture has two main layers (layer1 and layer2), each made up of several residual blocks. The layers have different filter sizes to capture features at

different scales. The output of the last layer is flattened and sent to a fully connected layer (fc1) for classification.

Loading training and validation data: Training and validation data are loaded from image and CSV label files using the CustomDataset class. The data is pre-processed using transformations such as image resizing and tensor conversion.

Initialisation of the model, loss function and optimiser: A ResNet object is instantiated with the specified parameters (number of blocks, number of classes, etc.). The CrossEntropyLoss function is used to calculate the loss between model predictions and actual labels. The Adam optimizer is used to update model weights during training.
Model training: The model is trained on the training data using a training loop. During each epoch, images are propagated through the network, predictions are compared with actual labels and the loss is calculated. The optimizer updates the model weights to minimise the loss.

Model evaluation on the validation set: After each epoch, the model is evaluated on the validation set to monitor its performance. Metrics such as loss, accuracy, precision, recall and F1 score are calculated to assess the model's performance.
This code illustrates the implementation of a ResNet model for image classification and its training on training and validation datasets.

3 MedMNIST

MedMNIST marks a pivotal advancement in utilizing machine learning for medical imaging analysis. Unlike traditional datasets focused on simple tasks like digit recognition, MedMNIST encompasses a broad spectrum of medical scans and images, aiming to boost the development and evaluation of models for disease classification, image segmentation, and anomaly detection. Its comprehensive and diverse collection of preprocessed medical images simplifies access for researchers, significantly aiding in the application of sophisticated machine learning solutions to healthcare challenges.

3.1 Focus on PathMnist

Within the MedMNIST collection, PathMNIST serves as a focused subset that specifically addresses colon pathology. This dataset is designed to support multi-class classification tasks with nine distinct classes or labels, reflecting the diverse nature of pathological findings that can be observed in colon tissue samples.

- **Dataset Composition:** Boasting 107,180 samples split into training, validation, and test sets with counts of 89,996, 10,004, and 7,180 respectively, PathMNIST is designed to provide a rich framework for model training across a broad image spectrum, critical for developing robust, generalizable models.
- **Data Modality and Tasks:** Focusing on colon pathology, PathMNIST challenges machine learning models with histopathological images, an area marked by intricate tissue details and variations. The task requires identifying subtle distinctions among nine types of pathological findings, pushing models to achieve nuanced pattern recognition.
- **Importance for Medical Research:** PathMNIST plays a pivotal role in advancing machine learning applications within medical diagnostics, particularly in colon pathology. It offers a benchmark for algorithm development and evaluation, aiming to enhance the efficiency and accuracy of disease diagnosis, thus impacting patient outcomes significantly.

```python
import os
import numpy as np
from PIL import Image

data_path = "/content/drive/MyDrive/kaggle/pathmnist_shuffled_kaggle.npz"
medmnist_data = np.load(data_path)
images = medmnist_data['train_images_shuffled']
labels = medmnist_data['train_labels_shuffled']
labels = [item for sublist in labels for item in sublist]
num_images = images.shape[0]
output_dir = "/content/output_images"
os.makedirs(output_dir, exist_ok=True)

for i in range(num_images):
    image_array = images[i].squeeze()
    label = labels[i]
    image_array = (image_array * 255).astype(np.uint8)
    image_pil = Image.fromarray(image_array)
    image_filename = f"{output_dir}/image_{i}_label_{label}.png"
    image_pil.save(image_filename)

print(f"Images saved to: {output_dir}")
```

Figure 3: Preprocessing.

3.2 Code Implementation:

Data Loading and Preprocessing:

- The code loads shuffled images and labels from a specified path using NumPy's np.load function, then extracts them. The nested list of labels is flattened into a single list using list comprehension for easier manipulation later.

Image Saving:

- The code iterates over each image in the dataset using a for loop.
- For each image, it converts the image array to a PIL Image object (Image.fromarray), saves it as a PNG file (image_pil.save), and names the file based on its index and label.
- The images are saved to a directory named output_images_res in the specified path (output_dir).

Output Directory:

- If the output directory doesn't exist (os.makedirs), it's created using os.makedirs.

Image Filenames:

- It creates a list of filenames for the saved images using a list comprehension. Each filename includes the image index and its corresponding label.

4 Convolutional Neural Networks (CNNs) Introduction

Convolutional Neural Networks (CNNs) are a class of deep neural networks, highly effective for analyzing visual imagery.They have been widely adopted in various applications, including image and video recognition, image classification, medical image analysis, and many others. The architecture of a CNN is designed to automatically and adaptively learn spatial hierarchies of features from input images.

The core building blocks of a CNN include convolutional layers, pooling layers, and fully connected layers:

- **Convolutional Layers:**

  Convolutional layers apply filters to input images, extracting features like edges and textures through a dot product operation, producing feature maps that highlight these features.

- **Pooling Layers:**

  Pooling layers reduce feature map dimensionality, cutting down computation and parameters. They help achieve feature invariance to scale and orientation, often using max pooling to select the maximum value within a filter region.

- **Fully Connected Layers:**

  In CNNs, fully connected layers aggregate all features to identify complex patterns, similar to traditional neural networks, and classify images into categories based on these learned features.

- **Activation Functions:**

  ReLU (Rectified Linear Unit) is a key activation function in CNNs, introduced post-convolution to add non-linearities, allowing the network to learn complex patterns.

- **Output Layer:** The softmax layer typically serves as the CNN's output layer for classification tasks, calculating the probability of the image belonging to each class.

```python
class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, stride=1, padding=1)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)

        self.conv2 = nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3, stride=1, padding=1)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)

        self.fc1 = nn.Linear(64 * 7 * 7, 128)
        self.relu3 = nn.ReLU()
        self.fc2 = nn.Linear(128, num_classes)

    def forward(self, x):
        x = self.conv1(x)
        x = self.relu1(x)
        x = self.pool1(x)

        x = self.conv2(x)
        x = self.relu2(x)
        x = self.pool2(x)

        x = x.view(x.size(0), -1)

        x = self.fc1(x)
        x = self.relu3(x)
        x = self.fc2(x)

        return x
```

Figure 4: Training[1].

4.1 Code Implementation:

Initialization Method (__init__):

- This method initializes the neural network architecture. num_classes=9: It takes an optional argument num_classes, which specifies the number of classes for classification. Default is set to 9.

Convolutional Layers:

- **self.conv1:** First convolutional layer with 3 input channels (for RGB images), 32 output channels, a kernel size of 3x3, stride of 1, and padding of 1.

- **self.relu1:** ReLU activation function applied after the first convolutional layer.

- **self.pool1:** Max pooling layer with a kernel size of 2x2 and stride of 2 after the first convolutional layer.

- **self.conv2:** Second convolutional layer with 32 input channels (output channels of the previous layer), 64 output channels, a kernel size of 3x3, stride of 1, and padding of 1.

- **self.relu2:** ReLU activation function applied after the second convolutional layer.

- **self.pool2:** Max pooling layer with a kernel size of 2x2 and stride of 2 after the second convolutional layer.

Fully Connected Layers:

- **self.fc1:** First fully connected layer with an input size calculated as 64 channels * 7 * 7 (output size of the second convolutional layer), and an output size of 128.

- **self.relu3:** ReLU activation function applied after the first fully connected layer.

- **self.fc2:** Second fully connected layer with an input size of 128 and an output size equal to the number of classes (num_classes).

Forward Method (forward):

- This method defines the forward pass of the neural network.

- It sequentially passes the input tensor x through each layer defined in the initialization method.

- After passing through the convolutional and pooling layers, the tensor is reshaped (x.view(x.size(0), -1)) to be compatible with the fully connected layers.

- Finally, the tensor is passed through the fully connected layers to get the output logits for each class.

```
num_epochs = 15
epoch_loss=0
model = SimpleCNNMultiClassReLU(num_classes=9)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
train_losses = []
val_accuracies = []
for epoch in range(num_epochs):
    model.train()
    for batch in train_loader:
        inputs, labels = batch['image'], batch['label']
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()
    average_train_loss = epoch_loss / len(train_loader)
    train_losses.append(average_train_loss)
    model.eval()
    with torch.no_grad():
        total_correct = 0
        total_samples = 0
        for batch in val_loader:
            inputs, labels = batch['image'], batch['label']
            outputs = model(inputs)
            _, predicted = torch.max(outputs, 1)
            total_correct += (predicted == labels).sum().item()
            total_samples += labels.size(0)

        accuracy = total_correct / total_samples
        val_accuracies.append(accuracy)
        print(f'Epoch {epoch + 1}/{num_epochs}, Validation Accuracy: {accuracy:.4f}')
```

Figure 5: Training[2].

This code conducts training and validation of a convolutional neural network (CNN) model for a multi-class classification task using PyTorch. It trains the model over a

specified number of epochs, computing the loss and updating the model's parameters using the Adam optimizer. During training, it accumulates the average training loss for each epoch. After each epoch, the model's performance is evaluated on a validation set to compute the validation accuracy. The training loop iterates through batches of training data, while the validation loop evaluates the model's performance on batches of validation data. This process is repeated for the specified number of epochs, and the training loss and validation accuracy are recorded for each epoch. Overall, the code orchestrates the training and evaluation process of the CNN model for the given classification task.

5 Introduction to Residual Networks (ResNet)

Residual Networks, or ResNet, represent a seminal innovation in deep learning architectures, specifically designed to address the challenges of training very deep neural networks. As networks grow deeper, they tend to suffer from vanishing and exploding gradient problems, which can hinder the training process and degrade performance. ResNet, introduced by Kaiming He et al. in their 2015 paper "Deep Residual Learning for Image Recognition," revolutionizes the architecture of deep neural networks through the introduction of "residual blocks" to facilitate the training of networks that are substantially deeper than those previously used.

5.1 Core Concept of ResNet

ResNet revolutionizes deep learning by introducing "shortcut connections" that bypass one or more layers, allowing inputs to be added directly to deeper layer outputs. This mechanism facilitates gradient flow, addressing the vanishing gradient issue and enabling deeper network training.

5.2 Residual Blocks

At ResNet's heart are residual blocks, featuring an "identity shortcut connection" for direct input forwarding and a secondary path through two or three convolutional layers. Outputs from these paths are combined, helping the network learn "residual functions" for easier optimization and enhanced accuracy at greater depths.

5.3 Variants of ResNet

ResNet variants, such as ResNet-18 through ResNet-152, differ in depth. Deeper versions employ "bottleneck" structures with a 1x1 convolution for dimensionality management and a 3x3 convolution for feature processing, optimizing computational efficiency.

```python
class ResNet(nn.Module):
    def __init__(self, block, layers, num_classes=9):
        super(ResNet, self).__init__()
        self.in_channels = 32
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, stride=1, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(32)
        self.relu = nn.ReLU(inplace=True)
        self.layer1 = self.make_layer(block, 64, layers[0], stride=2)
        self.layer2 = self.make_layer(block, 128, layers[1], stride=2)
        self.fc1 = nn.Linear(128 * 7 * 7, 128)
        self.relu3 = nn.ReLU()
        self.fc2 = nn.Linear(128, num_classes)

    def make_layer(self, block, out_channels, blocks, stride=1):
        layers = []
        layers.append(block(self.in_channels, out_channels, stride))
        self.in_channels = out_channels
        for _ in range(1, blocks):
            layers.append(block(out_channels, out_channels))
        return nn.Sequential(*layers)

    def forward(self, x):
        out = self.conv1(x)
        out = self.bn1(out)
        out = self.relu(out)
        out = self.layer1(out)
        out = self.layer2(out)
        out = out.view(out.size(0), -1)
        out = self.fc1(out)
        out = self.relu3(out)
        out = self.fc2(out)
        return out
```

Figure 6: Resnet.

6 Implementation of Code

In the initialization method (__init__), the model architecture is set up:

- The initial number of input channels is set to 32.
- The first convolutional layer (self.conv1) takes input with 3 channels (typical for RGB images), performs convolution with 32 output channels, and uses a 3x3 kernel size with a stride of 1 and padding of 1. It has no bias term.
- Batch normalization (self.bn1) and ReLU activation are applied after the first convolutional layer.
- The model consists of two sets of residual layers (self.layer1, self.layer2), each created using the make_layer method. This method constructs a series of residual blocks based on the specified block type, out_channels, blocks, and stride.
- Two fully connected layers (self.fc1, self.fc2) are defined for classification. self.fc1 takes the flattened output from the last residual layer and maps it to a 128-dimensional space, followed by a ReLU activation (self.relu3). Finally, self.fc2 maps the 128-dimensional space to the number of output classes (num_classes).

The make_layer method constructs each layer of the ResNet by creating a series of residual blocks. The forward method defines the forward pass of the model, passing the input tensor through the defined layers and returning the final output logits for each class. Overall, this code constructs a ResNet model suitable for image classification tasks.

7 Implementation of Residual Blocks

In the context of enhancing deep learning models, particularly for complex tasks such as analyzing medical images in the PathMNIST dataset, implementing residual blocks,

as inspired by the ResNet architecture, plays a pivotal role in the model's performance. Residual blocks are designed to tackle the degradation problem inherent in deep neural networks, where adding more layers leads to higher training error, not because of overfitting, but due to the increased complexity in optimizing the network's parameters.

7.1 Structure of a Residual Block

A residual block, the cornerstone of the ResNet architecture, comprises two main paths. The primary path carries out the conventional processing through convolutional layers, including the application of filters, activation functions, and potentially batch normalization. The distinctive feature of a residual block is its second path, the shortcut or skip connection, that bypasses one or more layers.

In a typical residual block designed for a deep neural network:

- **T**he input passes through two or more convolutional layers, each followed by batch normalization and a ReLU activation function.
- **S**imultaneously, the input is also forwarded directly down the network via the shortcut connection, skipping these convolutional operations.
- **T**he outputs from both paths are then added together before being passed to the next layer or block in the network.

7.2 Benefits of Using Residual Blocks

- **Ease of Training:** By allowing gradients to flow through the shortcut connections, residual blocks help alleviate the vanishing gradient problem, making it easier to train deep networks.
- **Improved Performance:** Networks equipped with residual blocks can learn more complex patterns without the degradation problem, leading to improved performance on a wide range of tasks.
- **Flexibility and Scalability:** Residual blocks can be easily integrated into existing architectures, providing a modular approach to increasing network depth and complexity without a proportional increase in training difficulty.

```python
class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1):
        super(ResidualBlock, self).__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU(inplace=True)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, stride=1, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)

        self.shortcut = nn.Sequential()
        if stride != 1 or in_channels != out_channels:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=stride, bias=False),
                nn.BatchNorm2d(out_channels)
            )

    def forward(self, x):
        residual = x
        out = self.conv1(x)
        out = self.bn1(out)
        out = self.relu(out)
        out = self.conv2(out)
        out = self.bn2(out)
        out += self.shortcut(residual)
        out = self.relu(out)
        return out
```

Figure 7: Residual Block.

7.3 Code Implementation:

Within the initialization method:

- Two convolutional layers (self.conv1, self.conv2) are defined, each followed by batch normalization (self.bn1, self.bn2). These convolutional layers have 3x3 kernels, and their strides are determined by the stride parameter.
- The self.shortcut attribute is defined as an empty nn.Sequential() container initially. It will contain a shortcut connection if the input and output dimensions of the residual block are different (i.e., if stride != 1 or in_channels != out_channels).
- If the input and output dimensions differ, a convolutional layer and batch normalization are added to self.shortcut to adjust the dimensions accordingly.

Forward Method (forward):

- The forward method defines the forward pass of the residual block.
- The input tensor x is passed through the convolutional layers, batch normalization, and ReLU activation functions.
- The residual variable is assigned the value of x, capturing the original input to be added later.
- The output of the convolutional layers and the shortcut connection (if applicable) are added together (out += self.shortcut(residual)). This is the key residual connection that allows gradients to flow more directly during training.
- Finally, the ReLU activation function is applied to the output, and the resulting tensor is returned.

8 Results and Discussion

We used five different metrics to analyze and present our results.
Accuracy: Measures the ratio of images predicted correctly over the total number of instances.
Loss: Measures the model's prediction error. It gives an idea of how well the model is performing by comparing the predictions to the actual values.
Precision: Measures the correctly predicted positive instances to the total predicted positive instance which gives an idea about the accuracy of positive predictions.
Recall: Measures the correctly predicted positive instances to the total actual positive instances which gives an idea about all the captured positive instances.
F1 score: Measures the mean of recall and precision which is used to provide a balance between classes.
We also present a confusion matrix for each of our datasets. A confusion matrix is a table that evaluates the performance of an algorithm on a dataset for which the true values are known. The matrix uses the values True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). With these values, we can derive the performance metrics mentioned above.

8.1 ResNet MedMNIST

This section presents the results of the ResNet model on the MedMNIST dataset. The distribution graph shows that the predicted values match closely with the actual distribution provided for the dataset on Kaggle. In real-world scenarios, an actual distribution is not available because that might influence the model to fit the actual data. However, in an academic setting, it is interesting to see such a result and determine how the model is predicting compared to actual data.

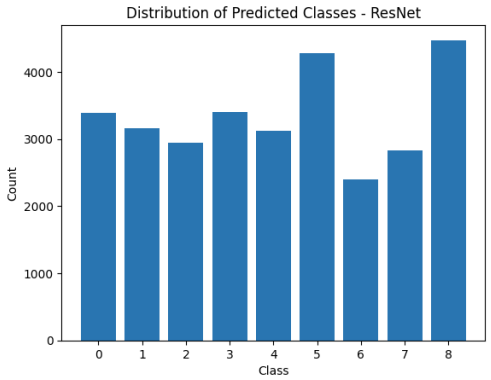


Figure 8: Predicted distribution Resnet

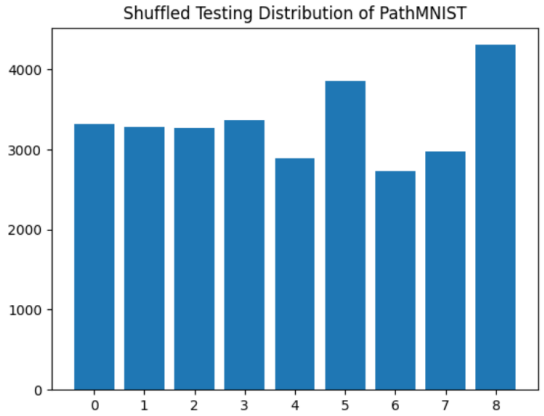


Figure 9: Actual distribution Medmnist

Figure 10 'Loss and Accuracy - Resnet Medmnist' shows that the training loss is consistently decreasing over the epochs which indicates that the model is improving to minimize the training sets error. The validation loss fluctuates but generally decreases which is an indication that the model is not overfitting and it is generalizing well to the validation set. On the other hand, the training accuracy increases over the epochs which means that the model is learning well from the training data. The validation

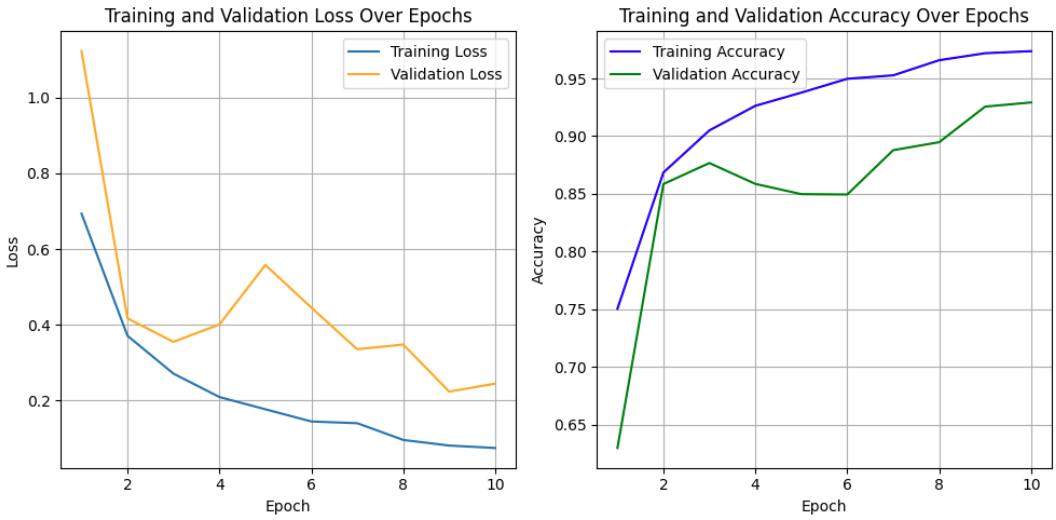


Figure 10: Loss and Accuracy - Resnet Medmnist

accuracy is also increasing meaning that the model is generalizing well to the unseen validation data. Since both the training and validation accuracy are increasing over the epochs it is a good sign that the model improving and learning well over the epochs.

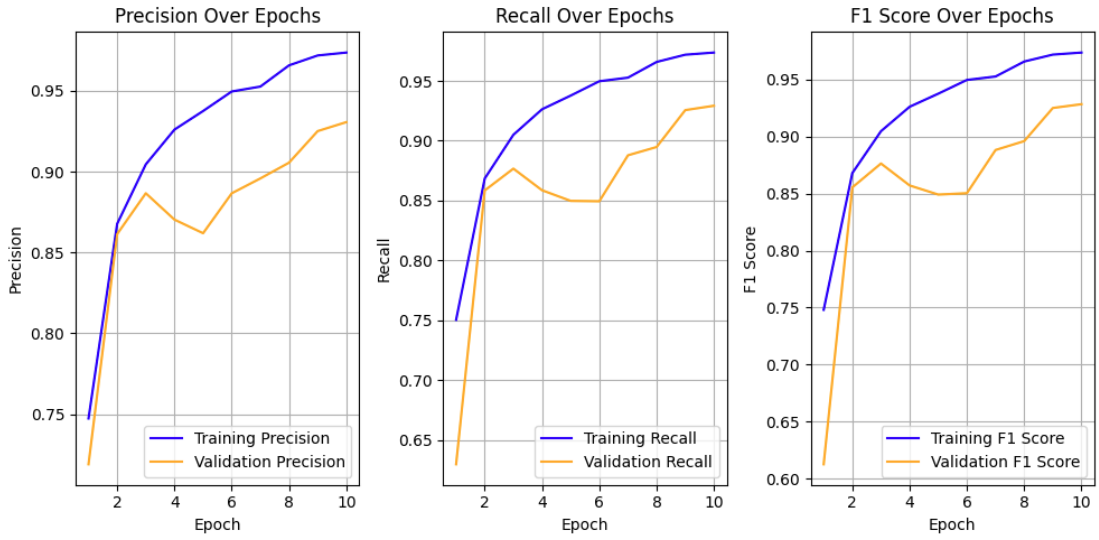


Figure 11: Precision, Recall, and F1 score - Resnet Medmnist

Figure 11 "Precision, Recall, and F1 score - Resnet Medmnist" shows that the training precision and validation precision increase because the model is getting better in training and identifying positive instances among predicted positive instances. The training recall is increasing consistently so the model is becoming better and better at identifying positive instances in the training set. The validation recall has a small dip but it is also increasing over epochs so the model is improving in generalizing unseen data. F1 score balances precision and recall therefore it is also increasing over epochs. All three of these have increasing patterns which indicate that the model is improving

in identifying positive instances. The trends in the model are not exhibiting clear signs of any overfitting or underfitting. Overfitting might have been detected if the validation metrics stopped or degraded and the training metrics kept increasing but such a behavior is not visible from the plots. The training and validation metrics have consistent patterns so they do not fit any noise in the training data.

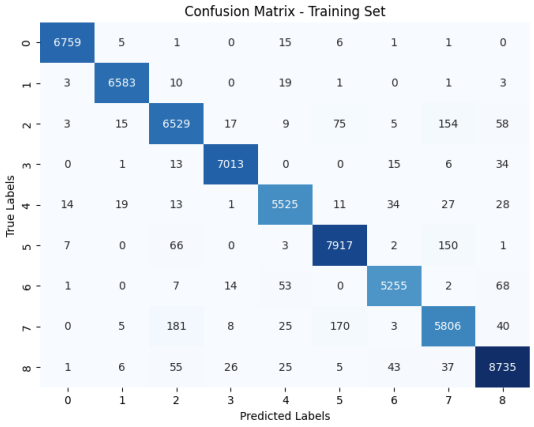


Figure 12: Confusion Matrix - Training - Resnet Medmnist

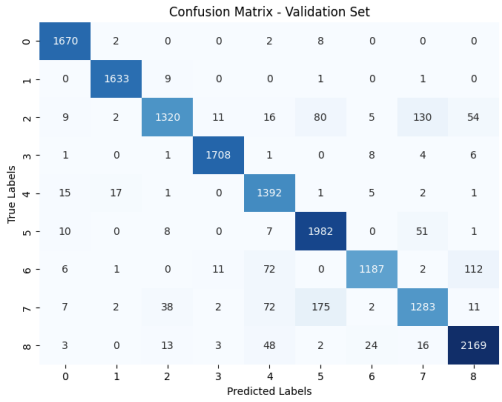


Figure 13: Confusion Matrix - Validation - Resnet Medmnist

The training and validation confusion matrices have high counts on the diagonal (which indicates true positive) meaning that the majority of instances are labeled correctly. The values away from the diagonals are less in quantity meaning that there is less misclassification. Most classes have high precision, recall, and F1 scores.

8.2 ResNet MNIST

This section presents the results of the ResNet model on the MNIST dataset. We will also compare it with the performance of Resnet on the MedMNIST dataset. The distribution of predictions shows that the data is split almost evenly between all the classes. We do not know the actual distribution of the MNIST dataset therefore an accurate comparison cannot be made until the testing dataset distribution is known.

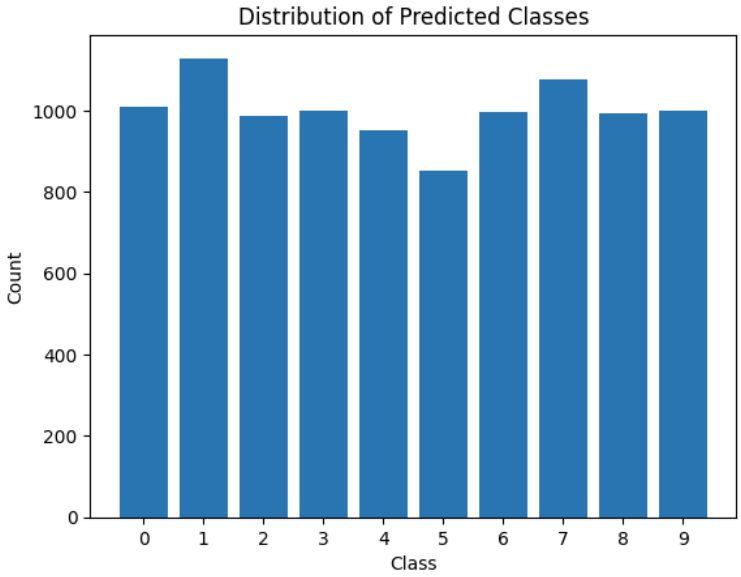


Figure 14: Distribution of predictions - Resnet Mnist

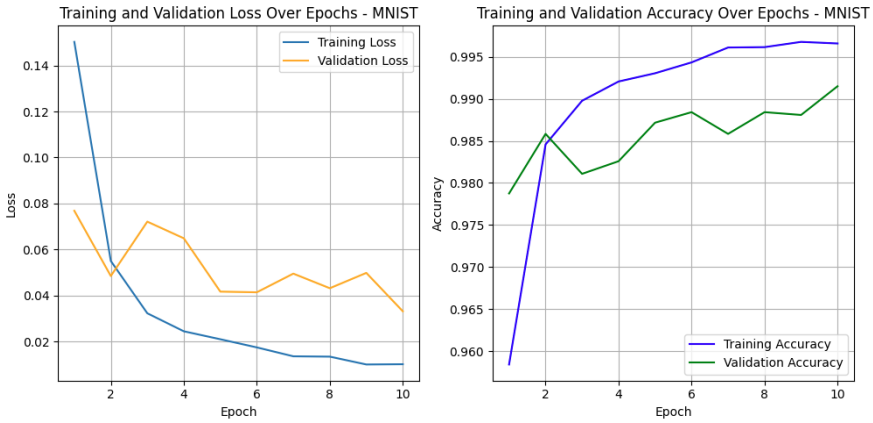


Figure 15: Loss and Accuracy - Resnet Mnist

Figure 15 'Loss and Accuracy - Resnet Mnist' shows that the training loss decreases consistently meaning the model is improving in making predictions on the training set. The validation loss is also following a decreasing trend, which suggests that the model is generalizing well to unseen validation data. The training accuracy increases quite fast in the beginning but the slope of increase becomes lesser after 4 epochs. The validation accuracy is high despite having some dips around the 3rd, 7th, and

9th epochs. However, it also continues to increase so the model generalizes well to the validation dataset.

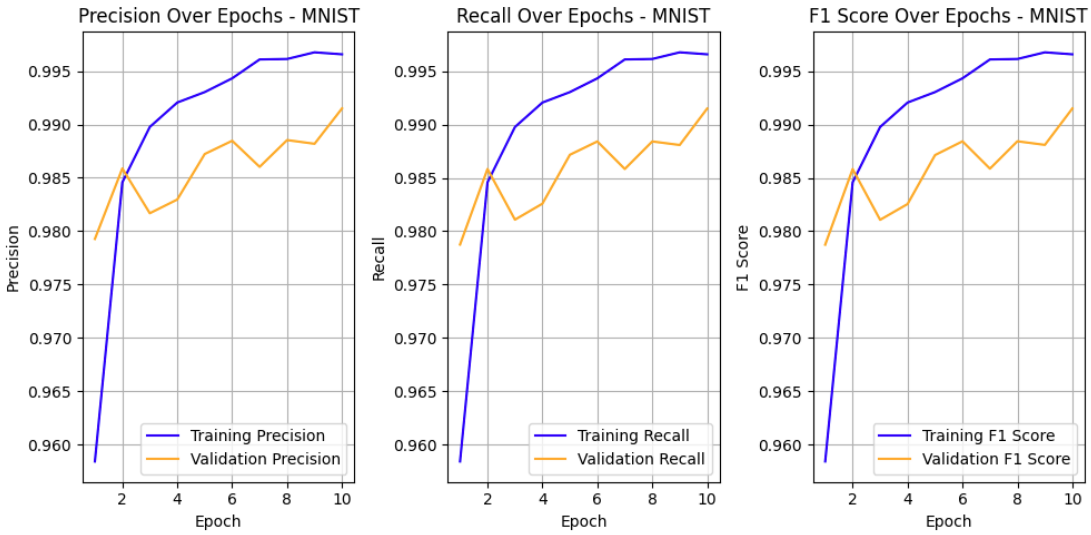


Figure 16: Precision, Recall, and F1 score - Resnet Mnist

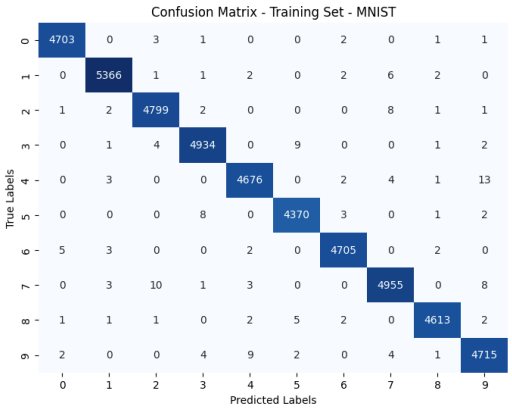


Figure 17: Confusion Matrix - Training Resnet Mnist

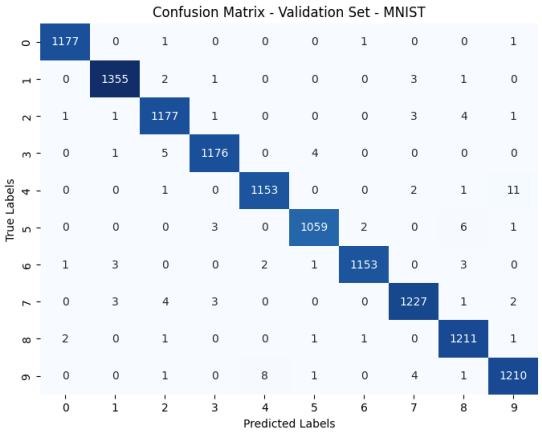


Figure 18: Confusion Matrix - Validation Resnet Mnist

Figures 17 and 18 'Confusion Matrix - Training Resnet Mnist' and 'Confusion Matrix - Validation Resnet Mnist' indicate that ResNet achieves extremely high performance with almost all instances being classified correctly. The high precision, F1 score, and recall show excellent model performance.

Resnet appears to perform very well on the MNIST dataset with high accuracy and low loss as compared to the trends observed in MedMNIST dataset. Overall resnet performs well on both MNIST and MedMNIST. ResNet achieves 99.15 percent validation accuracy and 99.66 percent training accuracy on MNIST compared to 92.93 percent validation accuracy and 97.37 percent training accuracy in MedMNIST. Resnet is outperforming on MNIST.

8.3 CNN MedMNIST

This section presents the results of the CNN model on the MedMNIST dataset. The predicted distribution is slightly different from the actual distribution acquired from Kaggle. The results are way off for class 2 but other class predictions are slightly accurate.

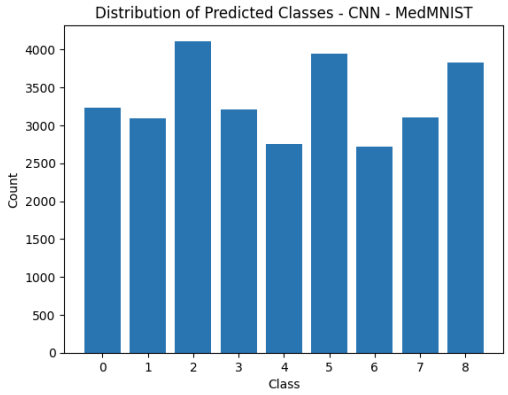


Figure 19: Predicted distribution - CNN Medmnist

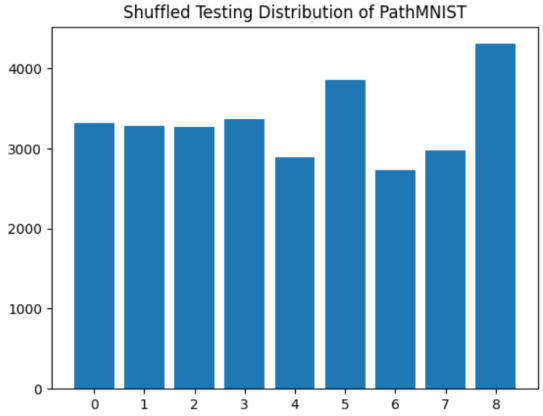


Figure 20: Actual distribution - CNN Medmnist

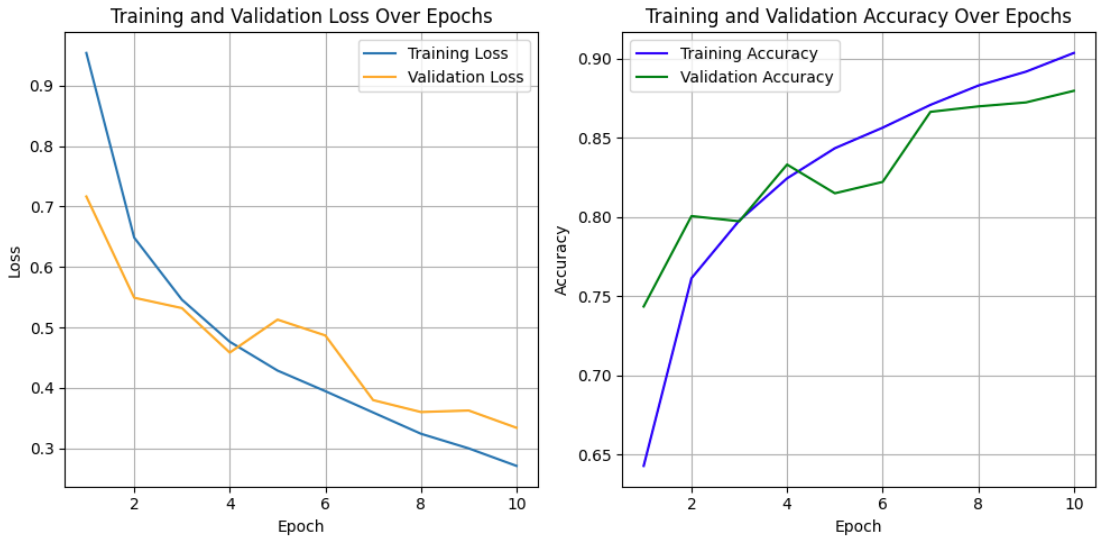


Figure 21: Loss and Accuracy CNN Medmnist

Figure 21 shows the loss and accuracy in the MedMNIST dataset with CNN. The training loss and validation loss are both decreasing implying that the model is converging and minimizing prediction errors. The validation loss is higher compared to the training loss. Accuracy is increasing for both training and validation meaning that the model is making correct predictions. When the model starts there is a relatively high training loss and low accuracy in the learning process. As training continues, the losses decrease indicating good convergence. The high precision suggests a low rate of misclassification for the predicted positives. In MedMNIST the recall rate is also high

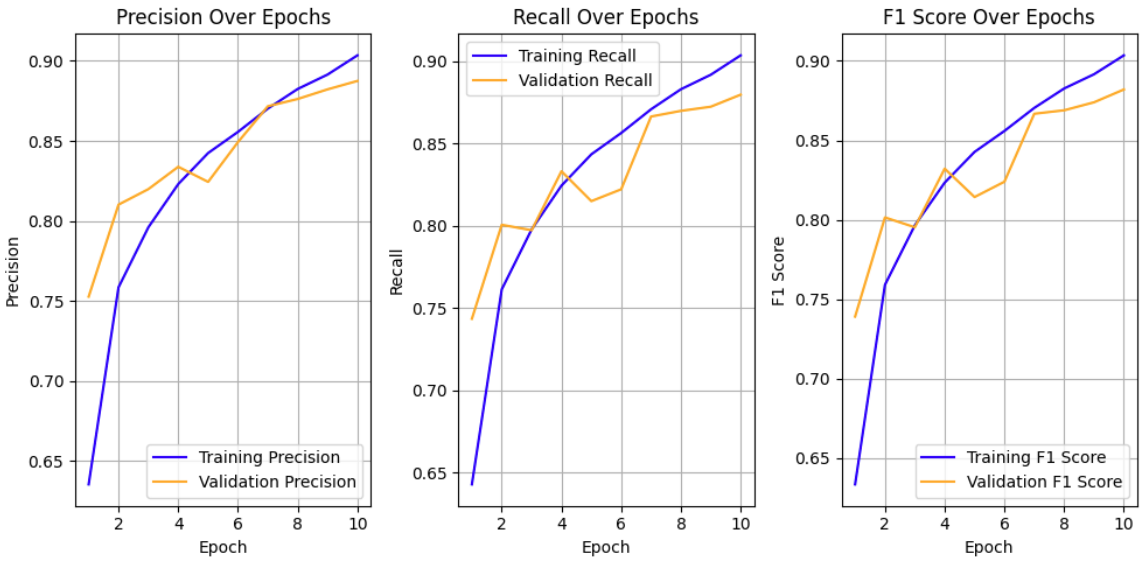


Figure 22: Precision, Recall, and F1 score CNN Medmnist

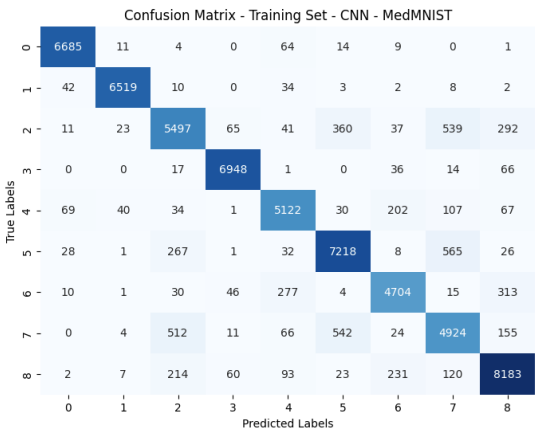


Figure 23: Confusion Matrix - Training CNN Medmnit

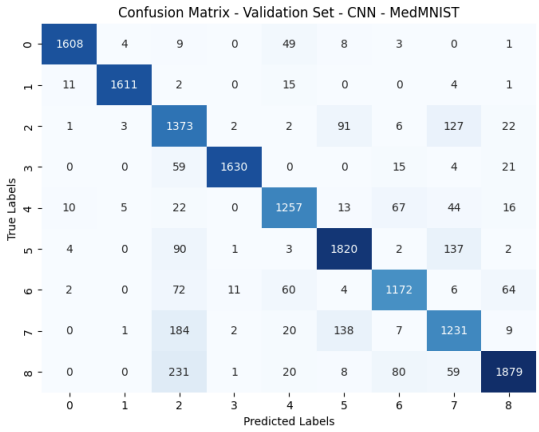


Figure 24: Confusion Matrix - Validation CNN Medmnist

in both training and validation with a good F1 score giving the high harmonic mean of precision and recall. Both the confusion matrices are showing high values on the diagonal indicating that the model is performing well. During training the accuracy is 90.37 percent which decreases to 87.98 percent during validations.

8.4 CNN MNIST

This section presents the results of the CNN model on the MNIST dataset and makes comparisons with CNN implementation on MedMNIST dataset.

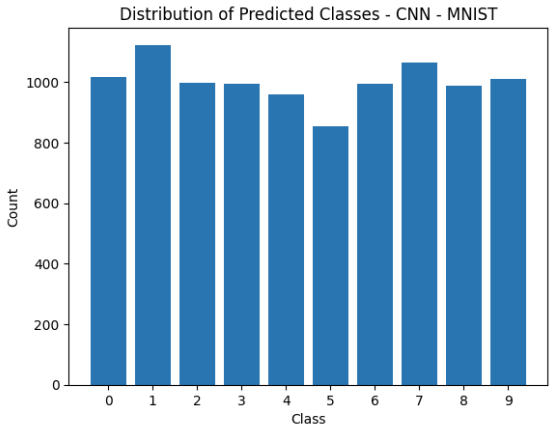


Figure 25: Distribution of predictions - CNN MNIST

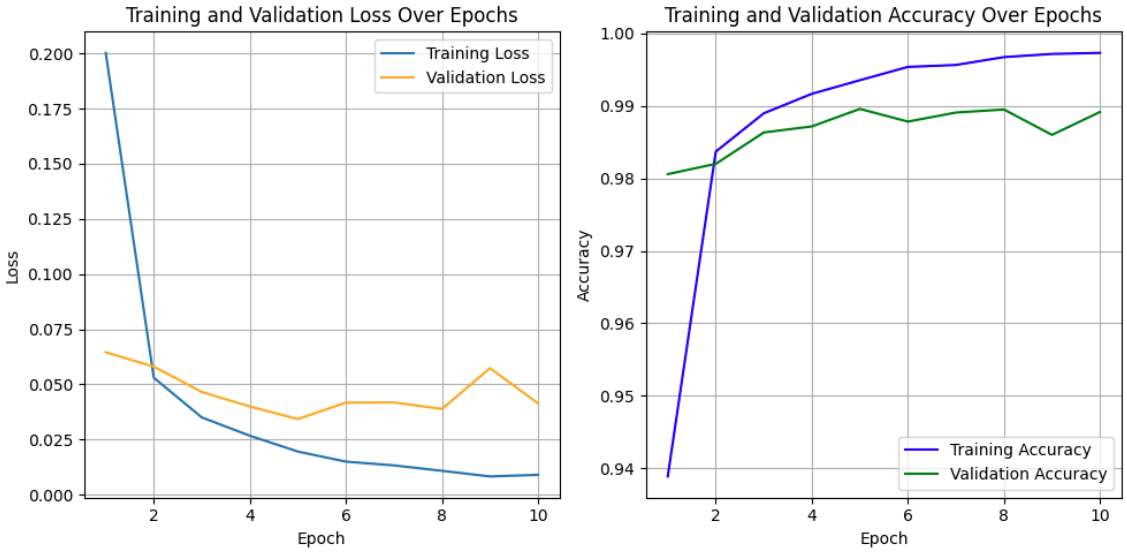


Figure 26: Loss and Accuracy - CNN MNIST

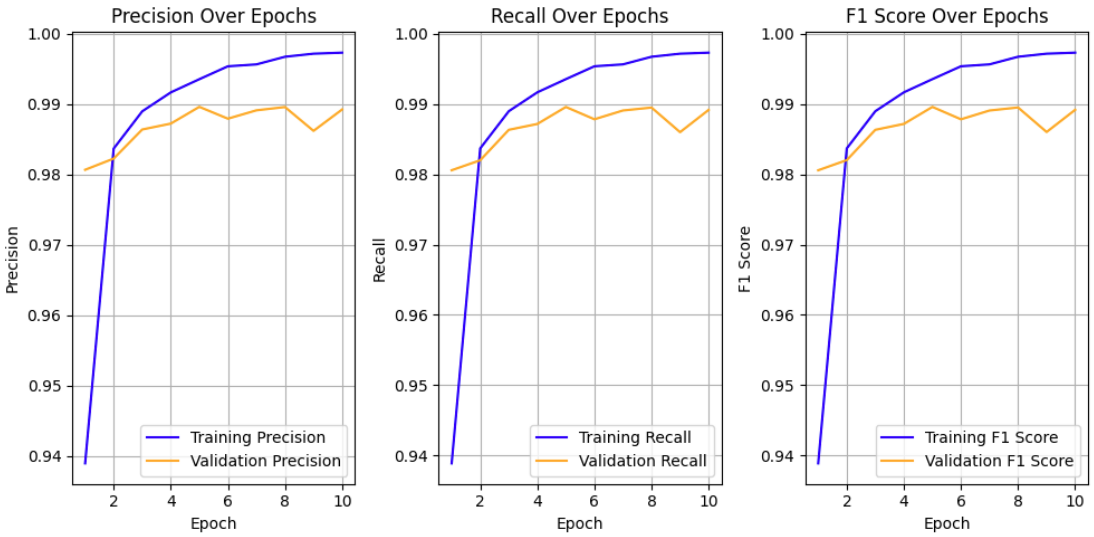


Figure 27: Precision, Recall, and F1 score - CNN MNIST

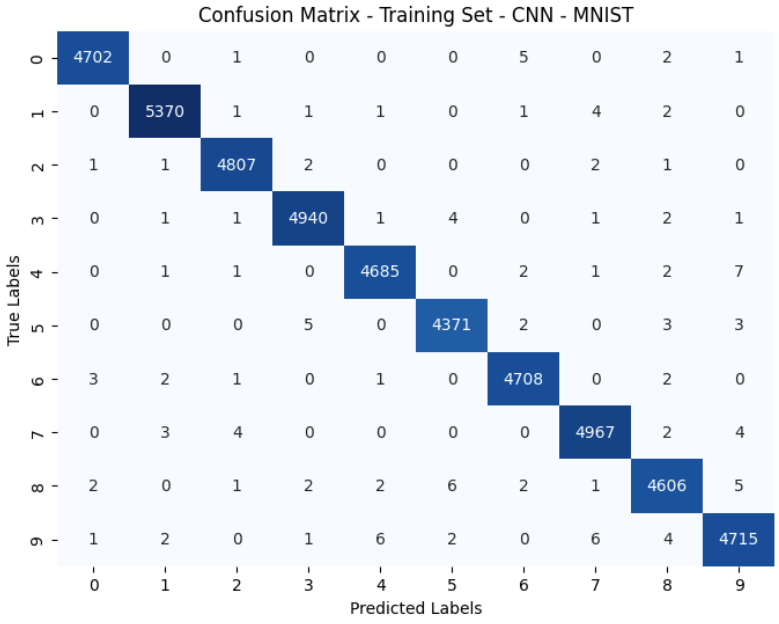


Figure 28: Confusion Matrix - Training - CNN MNIST

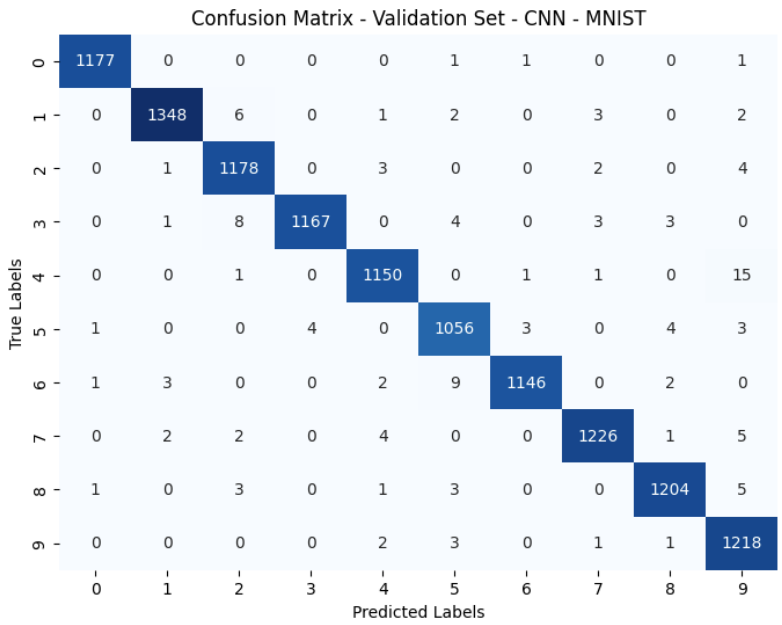


Figure 29: Confusion Matrix - Validation - CNN MNIST

With these results, we can observe that CNN is outperforming in MNIST compared to MedMNIST. In MNIST the training accuracy is 99.73 percent and the validation accuracy is 98.92 percent. In MedMNIST the training accuracy is 90.37 percent and validation accuracy is 87.98 percent. This performance difference could be attributed to the complexity of the MNIST dataset which is more well-established because of digit recognition whereas MedMNIST involves medical images which can have more variations. In MNIST a very high accuracy is achieved early in the training which indicates a quick convergence. MNIST dataset might have been easier for CNN to learn compared to MedMNIST. In MedMNIST we could see a larger drop in accuracy from training to validation which might be because of some overfitting.

8.5 Comparison of ResNet and CNN

The ResNet model achieved higher accuracy on both MNIST and MedMNIST datasets. There is a performance gap between training and validation sets which is more prominent in CNN on the Medmnist compared to ResNet on MedMnist. ResNet can better perform and generalize better on unseen data. ResNet architectures are deeper which provides a tradeoff between higher accuracy but increased computational complexity. MedMNIST consists of medical images which increases the complexity of the dataset. This complexity is reflected in the lower accuracy achieved by both CNN and ResNet on the MedMNIST dataset. In CNN we observed a larger drop in accuracy in MedMNIST with CNN. With ResNet there are less signs of overfitting compared to CNN in MedMNIST. This might be due to the residual connections in ResNet acting to mitigate the overfitting. CNN models were faster to converge compared to ResNet which needs more epochs to reach optimum performance.

9 Future Work

In future work, we plan to increase the number of training epochs and explore the effect on model convergence to view any performance improvement. We also intend to implement alternate architectures such as AlexNet or Xception to evaluate their effectiveness compared to Simple CNN and ResNet. We will experiment with different optimizers such as SGD, NAdam, and Adagram to identify the most suitable optimization technique for our models. Instead of ReLU, we could explore other activation functions such as Sigmoid. For the prevention of overfitting, we will implement early stopping techniques.

10 Conclusion

The project to manipulate the MNIST and MEDNIST datasets to create a predictive model was a rewarding experience that explored various aspects of machine learning and computer vision. Here are some key takeaways from the experience.

Manipulating the MNIST and MEDNIST datasets enabled us to familiarize ourselves with image pre-processing and data preparation for model training.Several models were subsequently explored, including convolutional neural networks (CNNs) and knowledge transfer models such as ResNet. Training these models has enabled us to understand the impact of model architectures and hyperparameters on prediction performance.Evaluating the performance of the models on the test and validation sets enabled us to measure their ability to generalize to new data. Analysis of metrics such as accuracy provided information on the quality of model predictions.Comparing the performance of different models, such as CNN and ResNet, has enabled us to understand the advantages and disadvantages of each approach. This comparison also helped guide the choice of the most appropriate model architecture for the prediction task.

11 GitLab Repositories

1: https://gitlab.studium.uni-bamberg.de/abeera-ayesha.riaz/xai-proj-b
2: https://gitlab.studium.uni-bamberg.de/ulrich-midrel.tchouli-nzeukeyo/xai-proj-b

References

[1] L. Bottou, C. Cortes, J. S. Denker, H. Drucker, I. Guyon, L. D. Jackel, Y. LeCun, U. A. Muller, E. Sackinger, P. Simard, *et al.*, "Comparison of classifier methods: a case study in handwritten digit recognition," in *Proceedings of the 12th IAPR International Conference on Pattern Recognition, Vol. 3-Conference C: Signal Processing (Cat. No. 94CH3440-5)*, vol. 2, pp. 77–82, IEEE, 1994.

[2] Y. LeCun, C. Cortes, C. Burges, *et al.*, "Mnist handwritten digit database," 2010.

[3] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.